Introduction to Algorithms and Data Structures

week 3.1: Recursion part 1

SAXION UNIVERSITY OF APPLIED SCIENCES

Discussing homework from the last lesson

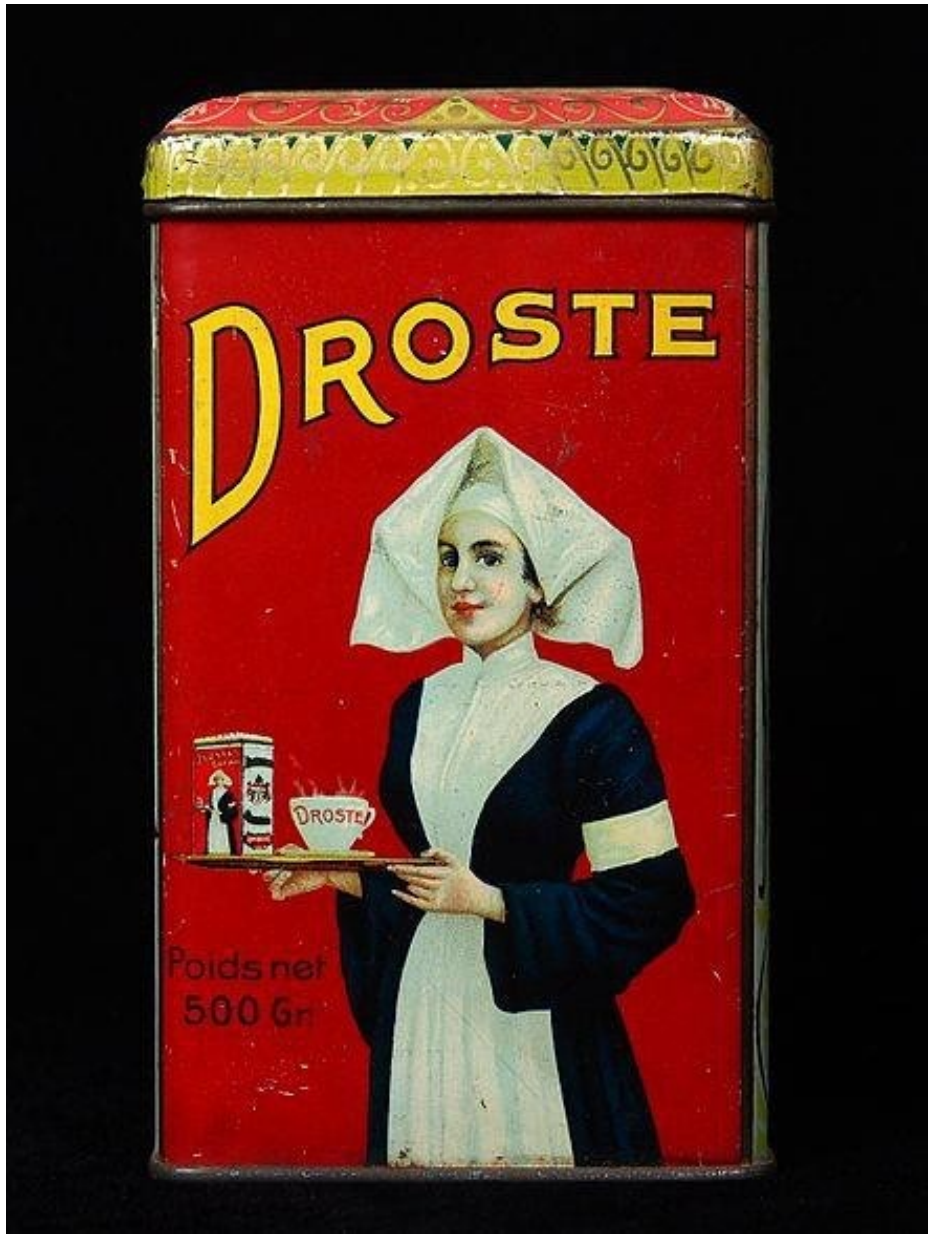
- Where you able to do the exercises?
  1. Yes! All exercises are done!
  2. No.., Because I have these questions
  3. I haven't prioritized this subject enough at the moment to do the assignments

Recursion

- **Discuss work last lesson**
- What is recursion
- homework

What is recursion?

- A function that solves a problem by calling the same function <u>again</u>
- In programming we try to solve a certain problem by making the problem smaller and try again, and again, and again.
  - At some point we no longer can make the problem smaller and we solve it

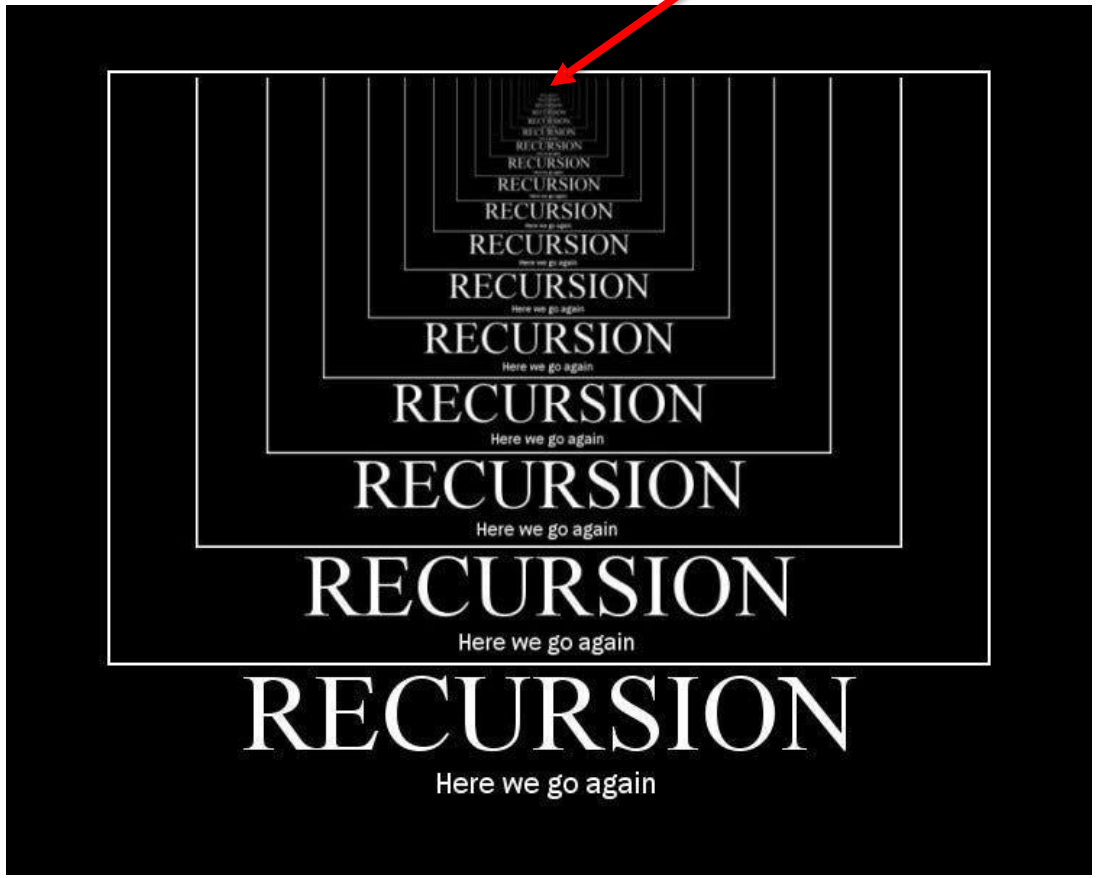


What is recursion?

*Hopefully some **base case***

- Recursion is a technique where a function solves a problem by:
  - **breaking it down** into smaller, similar sub-problems
  - **calls itself** to solve these sub-problems.

* This process continues iteratively until reaching a **base case** *



Base case: a problem small enough that it can be solved directly without further subdivision

Problem: How many people are in a line?

- Think of a solution to find out how many people are in front of you in a line, if
  - Each person can only see the person in front/behind of him/her
  - Each person can only speak to the person in front/behind of him/her

Ask yourself the following questions

1. When is the queue so small that you are absolutely sure how long the queue is?

2. What needs to happen if the queue is longer then 1 person?

Problem: How many people are in a line?

Ask yourself the following questions

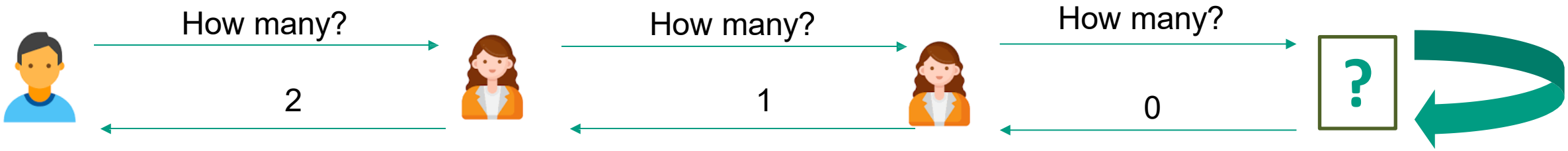
1. When is the queue so small that you are absolutely sure how long the queue is?

a queue of 1 person (base case)

2. What needs to happen if the queue is longer then 1 person?

ask the person in front and add 1 (recursive call)

Lets actually test this!



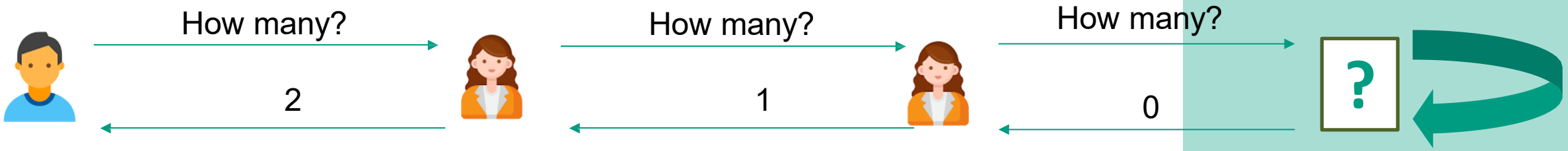
The one in front of me says 2 + me, that makes 3!

The one in front of me says 1 + me, that makes 2

The one in front of me says 0 + me, that makes 1

There is nobody in front of me!

Lets actually test this!



The one in front of me says 2 + me, that makes 3!

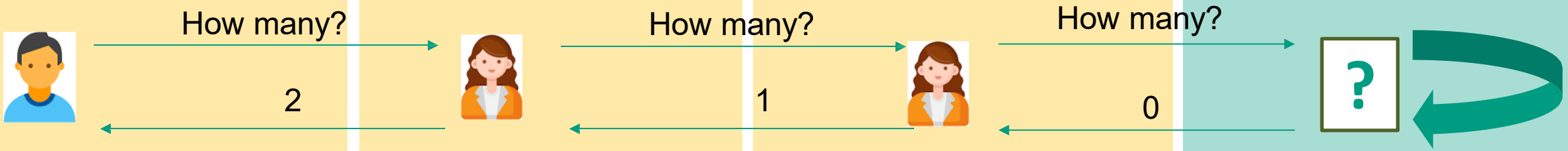
The one in front of me says 1 + me, that makes 2

The one in front of me says 0 + me, that makes 1

There is nobody in front of me!

Base case

Lets actually test this!



The one in front of me says 2 + me, that makes 3!

The one in front of me says 1 + me, that makes 2

The one in front of me says 0 + me, that makes 1

There is nobody in front of me!

Recursion level 0

Recursion level 1

Recursion level 2

Base case

Always try to make the problem smaller!

- Base cases solve the simplest problems, providing endpoints.

- Break down the problem into smaller, alike versions until reaching these base cases.

- Employ recursion when problems can be methodically simplified into smaller, identical forms, leading to resolution through repeated reduction.

- OR simpler: Use a recursive solution for problems that can be reduced to the same problem but smaller! *Until a base case is reached*

Let's build a recursive function together!

Sum positive numbers

Definition:

The sum of all positive integers not greater than **n**.

For example, calculate the sum of all number to 5. **n**=5.

Rewrite:

$\mathrm{Sum}(5) = 5 + 4 + 3 + 2 + 1 = 15$

$\mathrm{Sum}(5) = 5 + \mathrm{Sum}(4) = 15$

Rewritten to a sum function:

$\mathrm{Sum}(n) = n + \mathrm{Sum}(n\text{-}1)$ (recursive case)

$\mathrm{Sum}(0) = 0$ (base case)

Java code

```
private int sum(int number) {
    if (number == 0) {
        return 0;
    }
    return number + sum( number: number - 1);
}
```

Workshop:

Do exercise: 0_introductionToRecursion from **recursion**
Do exercise: 1_factorial from **recursion**

(Continue with the other exercises when you are done)

You can find this on BrightSpace > week3

Mini quiz

- What is the next number in this line?

$$\underbrace{0,\ 1,}_{\text{base cases}}\ \underbrace{1,\ 2,\ 3,\ 5,\ 8,\ 13,\ 21,\ 34,\ 55,\ \ldots}_{\text{recursion}}$$

Base case and recursive part

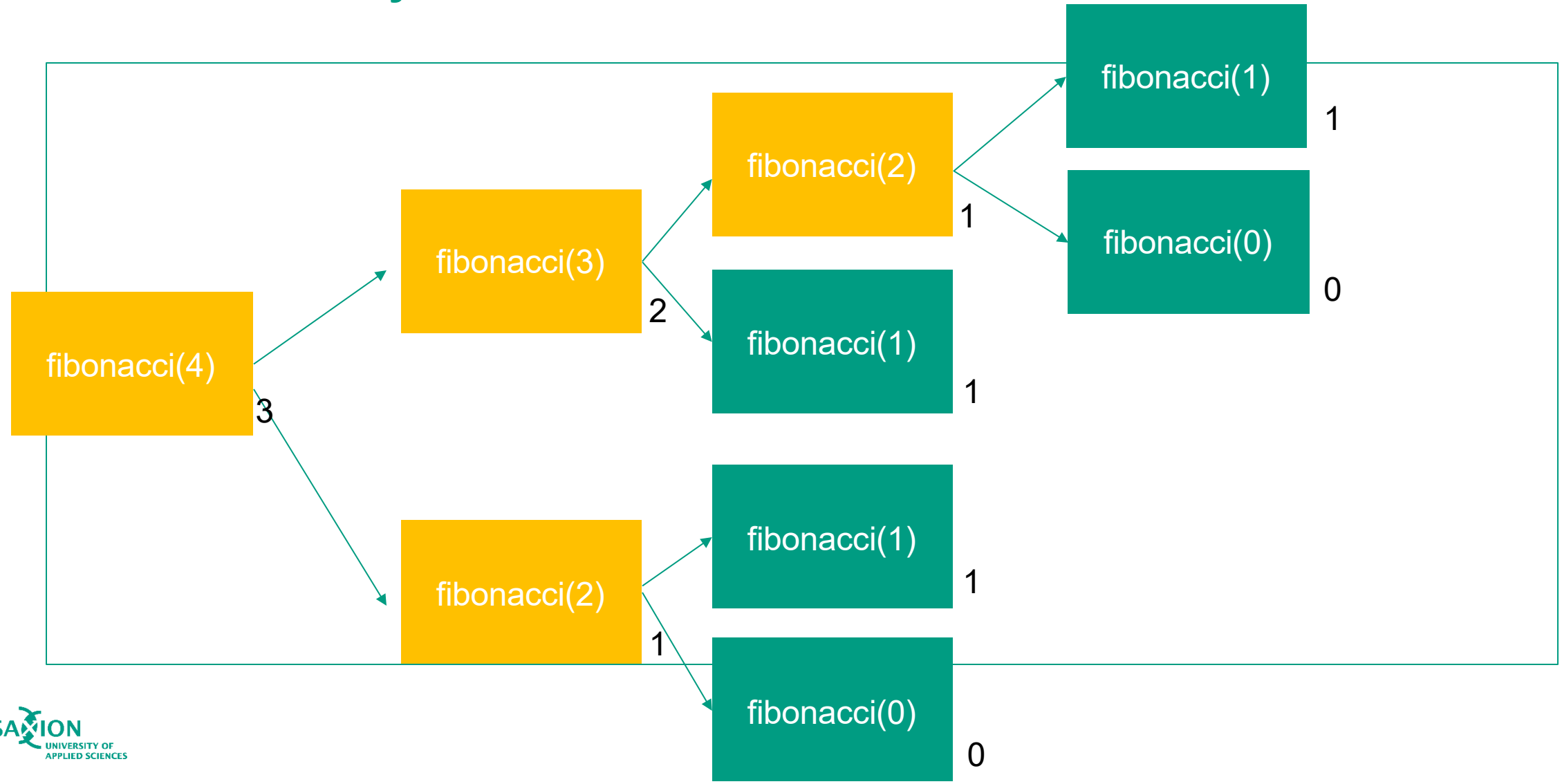
```
public int fibonacci(int count) {
    // TODO: implement

    // Base case(s)

    // Recursion part
```

Recursive call stack diagram

A recursive method will create a stack, that can lead to a stack overflow exception.

Let’s see how the stack looks like for the recursive factorial method, with $n=4$.

Recursion until you reach the base case!



Now it’s your turn!